

Fouad Bin Karim

Lab-09

5a) What is CircleDisplay() doing?

It initializes the program's data + UI layout so Swing can start painting.

b) When/how often does paintComponent() run?

paintComponent() runs whenever Swing decides the panel needs repainting, such as:

when the window first appears, when I resize / uncover the window, when code calls repaint()

c) What is JPanel (official docs)?

JPanel is a generic lightweight container in Swing used to group components and do custom drawing. It extends to JComponent.

d) Alter the color and size

Normally:

size = change the radius/diameter variable used in g.fillOval(...)

color = call g.setColor(new Color(...))

6) ContinuousLogisticGrowth (and InteractiveLogisticGrowth)

a) What term makes it different from exponential growth?

That term shrinks growth as the population approaches K.

b) Is it unpredictable/chaotic like the discrete cousin?

Usually no. The standard continuous logistic differential equation is smooth and predictable for typical parameters.

c) Does the population always converge?

For common “normal” settings (positive r , positive K , positive starting N), it typically converges toward K .

d) Does it ever collapse?

Yes, depending on parameters conditions

e) Does it ever explode to infinity?

No, the limiting term prevents runaway infinity for typical positive settings.

f) Compare to RabbitDiscreteSimulationGUI

Discrete sims can show:

oscillations

sensitivity to small changes

chaotic-looking behavior (depending on growth settings)

Continuous logistic tends to look:

smooth

stable

convergent (most of the time)

7) SierpinskiTriangle.java

a) How can it draw the same pattern at smaller scales?

Recursion: it draws a triangle, then calls itself to draw smaller triangles inside it, repeating until a base case.

b) What function is responsible for the shape of a particular triangle ?

the geometry math function that computes the triangle's vertices.

c) What function is responsible for the entire image?

The recursive function like `drawSierpinski(...)` / `sierpinski(...)` that repeatedly subdivides and calls itself.

d) Similarity to the discrete logistic map?

Both build complex outcomes by repeatedly applying a simple rule:

logistic map: iterate $x_{n+1} = f(x_n)$

sierpinski: recursively apply the “split triangle into smaller triangles” rule

8) `RandomHexPrinter` + `RawRandomHexStreamer` + `GetWebPage.java`

a) What are these programs doing?

`RandomHexPrinter`: prints some amount of random bytes formatted as hex

`RawRandomHexStreamer`: continuously streams random bytes as hex until I stop it (CTRL-C)

b) Where do they get randomness?

Usually from Java RNG utilities

c) Truly random?

`Random` / `Math.random()` are pseudorandom

`SecureRandom` is still technically pseudorandom, but designed to be cryptographically strong.

d) GetWebPage.java

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.net.URL;

public class GetWebPage {
    public static void main(String[] args) throws Exception {
        URL url = new URL("https://www.java.com/");
        try (BufferedReader in = new BufferedReader(new InputStreamReader(url.openStream())))
        {
            String line;
            while ((line = in.readLine()) != null) {
                System.out.println(line);
            }
        }
    }
}
```

9) CarrotGenerator.java

a) Random or pseudorandom?

Almost certainly pseudorandom.

b) What do the random values get used as?

Commonly:

carrot positions (x,y)

carrot sizes

c) How often does the timer execute?

10 times/sec, etc.

d) What code executes when the timer goes off?

The timer's ActionListener → its actionPerformed(...) method.

e) What variable tracks all carrots?

Usually something like ArrayList<Carrot> carrots;

10) RabbitsMover.java

a) What determines starting location?

Random within panel bounds, something like:

```
x = rand.nextInt(width); y = rand.nextInt(height);
```

b) What line initializes velocity?

when creating rabbit:

```
dx = rand.nextDouble()*...
```

```
dy = rand.nextDouble()*...
```

or a constructor call that sets velocity fields.

c) How many times/sec are positions updated?

Timer-based. If delay is 16 ms → ~60 fps. If 33 ms → ~30 fps.

d) What function updates position?

a loop inside the panel's timer tick.

e) How do they bounce off walls?

Boundary checks that reverse velocity, like:

if $x < 0 \parallel x > \text{width}$ then $dx = -dx$

same for y/dy

f) Make them really fast/slow

Either:

multiply velocities ($dx *= 5; dy *= 5;$)

11) WolfMover.java

a) How often update position?

Timer delay controls it.

b) What enables keyboard input?

`setFocusable(true);`

`addKeyListener(this);` (or a `KeyAdapter`)

c) What method does the timer execute?

`actionPerformed(...)` (`ActionListener`)

d) Code changes

faster: `int speed = 10;`

bigger: increase wolf size `int size = 40;`

color: change `g.setColor(...)` `g.setColor(Color.RED);`

invert controls: swap sign on velocity changes for each arrow key

```
if (key == KeyEvent.VK_LEFT) {
```

```
    x += speed; // LEFT now moves RIGHT
```

```
}
```

```
if (key == KeyEvent.VK_RIGHT) {
```

```
x -= speed; // RIGHT now moves LEFT
}
if (key == KeyEvent.VK_UP) {
    y += speed; // UP now moves DOWN
}
if (key == KeyEvent.VK_DOWN) {
    y -= speed; // DOWN now moves UP
}
```

12) SpriteLoader / SpriteLoader2 / SpriteAnimationPanel

a) Type holding image data?

Often BufferedImage

b) Where is the image and what is it?

in a resources/ folder inside the project

c) In SpriteLoader2, what type is loaded onto the window to be painted?

a JLabel holding an ImageIcon

d) Change loaded file to resources/spriteSheet.png

I saw the entire spritesheet grid instead of a single sprite.

e) Why is SpriteAnimationPanel animated?

Because it uses:

a timer loop

a changing frame index

repeated repaint()

13) MultispeciesContinuousSimulation (+ UI)

a) If graphing continuous equations, why no integration/differentiation?

Because it likely approximates using discrete time steps

b) What does each initial value represent?

initial populations (starting N)

growth rates (r)

carrying capacities (K)

interaction strengths

c) Why are curves smooth?

Because:

very small timestep updates

lots of points plotted close together

continuous-ish changes each tick

14) CarrotRabbitSimulation + ArcadeEcosystemSimulation

a) Main limiting factor on rabbit max size?

Food supply (carrots) + any artificial cap like maxPop.

b) Discrete or continuous logistic map?

Usually closer to continuous logistic-like behavior

c) How often carrots repopulate + max carrots?

Timer interval controls “how often,” and a constant controls “maximum carrots” (often something like MAX_CARROTS).

d) If you repopulate every 1 second, discrete or continuous?

More discrete

e)

i get rapid growth and then oscillations/bust cycles depending on how rabbits consume carrots and how births/deaths are implemented.

Extinction can happen if consumption outruns repopulation or if death logic is harsh.

f) ArcadeEcosystemSimulation: values that minimize difference vs the smooth graphs

small time steps

frequent small repop events

interaction rates tuned so the agent sim doesn't introduce extra noise